



Falconf: Configuration Error Diagnosis via Log Sequence Learning and Automated Misconfiguration Injection

Youyang Kim¹, Sahil Suneja², Yunja Choi¹,
Young-Woo Kwon¹, and Byungchul Tak¹

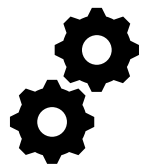
Kyungpook National University ¹ IBM TJ Watson Research Center ²

What is Configuration?

A configuration is a set of settings that controls system behavior.

System users set these parameter according to their needs.

- How much memory to allocate, which port to use, or where to store the data.



```
<property>
<name>hadoop.tmp.dir</name>
<value>/data/tmp</value>
<description>A base for other temporary directories.</description>
</property>

<property>
<name>fs.defaultFS</name>
<value>hdfs://namenode-host:9000</value>
<description>default hadoop host</description>
</property>

<property>
<name>hadoop.http.filter.initializers</name>
<value>org.apache.hadoop.http.lib.StaticUserWebFilter</value>
<description>A comma separated list of class names. Each class in the list
must extend org.apache.hadoop.http.FilterInitializer. The corresponding
Filter will be initialized. Then, the Filter will be applied to all user
facing jsp and servlet web pages. The ordering of the list defines the
ordering of the filters.</description>
</property>

<property>
<name>hadoop.http.idle_timeout.ms</name>
<value>60000</value>
<description>
NN/IN/DN Server connection timeout in milliseconds.
</description>
</property>

<property>
<name>hadoop.security.authorization</name>
<value>false</value>
<description>Is service-level authorization enabled?</description>
</property>

<property>
<name>hadoop.security.instrumentation.requires.admin</name>
<value>false</value>
<description>
Indicates if administrator ACLs are required to access
instrumentation servlets (JMX, METRICS, CONF, STACKS).
</description>
</property>
```

(a) Hadoop configuration file

```
...
Hadoop configuration
(core-site.xml)

<property>
  <name>hadoop.tmp.dir</name>
  <value>/tmp</value>
</property>

<property>
  <name>fs.defaultFS</name>
  <value>hdfs://namenode-host:9000</value>
</property>
...
```

(b) Part of the Hadoop configuration file

Misconfiguration is a Wrong Configuration Value

When one of the configuration parameters is set to a wrong value and causes the system to fail, we call it Misconfiguration.



```
... Hadoop configuration (core-site.xml)
<property>
  <name>hadoop.tmp.dir</name>
  <value> /tmp; </value>
</property>
...
```

*(a) Example of misconfiguration –
Path with invalid characters(;)*



```
... Hadoop configuration (core-site.xml)
<property>
  <name>hadoop.tmp.dir</name>
  <value> /var/tmp </value>
</property>
...
```

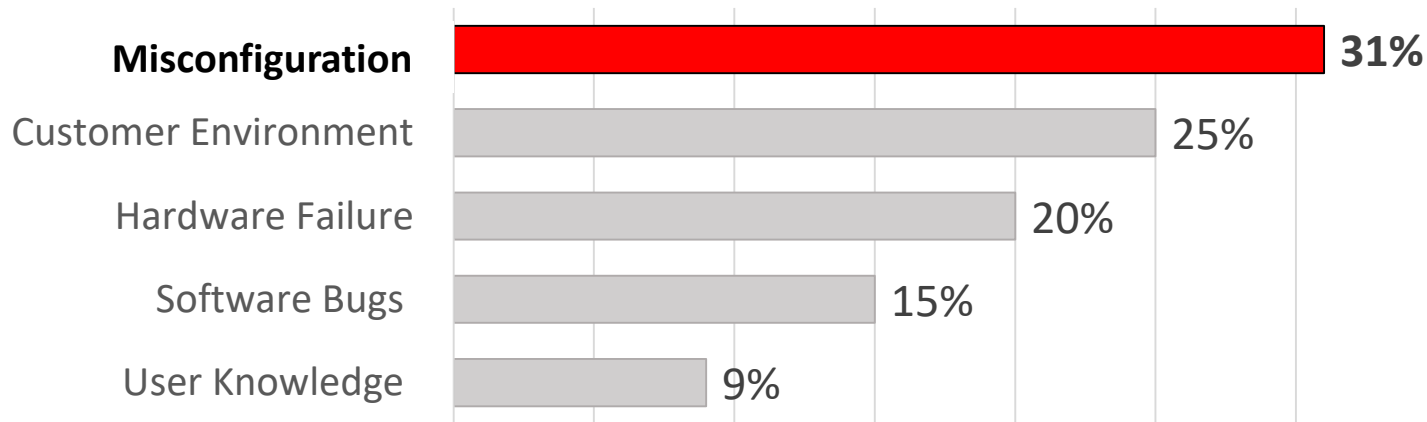
*(b) Example of misconfiguration –
non-existent path*

Misconfigurations are a Leading Cause of Failures

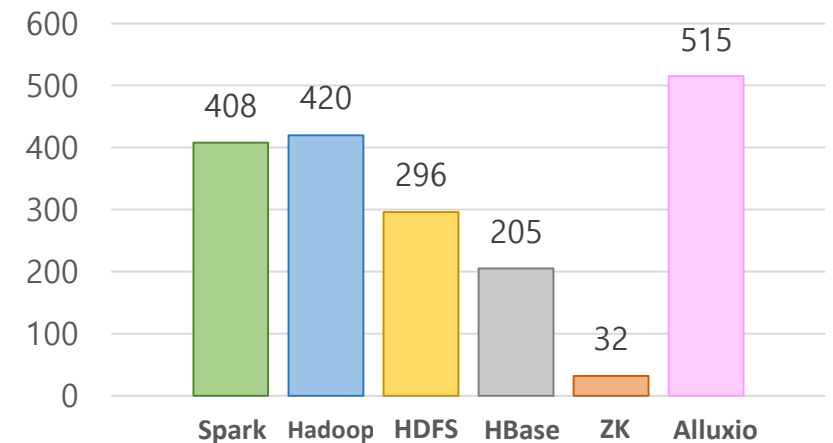
Misconfigurations are responsible for >30% of cloud/distributed system incidents.

Distributed systems have hundreds of configuration parameters, making it extremely difficult to identify which one caused the failure.

Root Causes of Cloud System Failure [1]



Number of Configuration Parameter [2]



- [1] Zuoning Yin, Xiao Ma, Jing Zheng, Yuanyuan Zhou, Lakshmi N. Bairavasundaram and Shankar Pasupathy, An Empirical Study on Configuration Error in Commercial and Open Source Systems, *SOSP'11 Proceeding of the Twenty-Third ACM Symposium on Operating Systems Principles*
- [2] Xudong Sun, Runxiang Cheng, Jianyan Chen, Elaine Ang, Owolabi Legunsen, and Tianyin Xu. Testing Configuration Changes in Context to Prevent Production Failures. *OSDI'20. Proceeding of the 14th USENIX Conference on Operating Systems Design and Implementation*

Syntactic vs Contextual Misconfigurations

We classify misconfigurations into two types.

- **Syntactic misconfigurations** – bad formats and wrong types
 - Built-in validators catch them at startup.
- **Contextual misconfigurations** - look correct but fail at runtime
 - No validator can catch them – far harder to detect.

*Bad format:
Path with semicolons*

```
<property>  
  <name>hadoop.tmp.dir</name>  
  <value> /tmp; </value>  
</property>
```

*Wrong type:
String value for
integer parameter*

```
<property>  
  <name>dfs.blocksize</name>  
  <value> 356 mb </value>  
</property>
```

(a) Example of syntactic misconfiguration

```
<property>  
  <name>hadoop.tmp.dir</name>  
  <value> /var/tmp </value>  
</property>
```

Non-existent Path

```
<property>  
  <name>dfs.blocksize</name>  
  <value> 10 </value>  
</property>
```

*Insufficient block
size value*

(b) Example of contextual misconfiguration

We Target Contextual Misconfiguration

Contextual Misconfigurations are harder to detect than syntactic misconfiguration because, ...

- No syntax error – passes all built-in validators
- No immediate symptoms at startup
- May manifest as silent failures with no error messages at all

We target Contextual misconfigurations.

Existing Approaches for Misconfiguration Diagnosis

Static Analysis

- Analyzes source code to detect invalid parameter types or missing options
- **✗** Can not incorporate runtime information
- [13'SOSP] SPEX, [20'FSE] cDEP

Dynamic Analysis

- Checks configuration validity at runtime through instrumentation
- **✗** High runtime overhead
- [08'DSN] ConfErr, [16'OSDI] PCheck

Knowledge-based Approaches

- Extract configuration specification from manuals, or match against knowledge base
- **✗** Rely heavily on external data sources, which are not always available in practice
- [15'VLDB] ConfSeer, [20'ATC] Pracextractor, [24'PLDI] Diffy

LLM-based Approaches

- Uses LLMs to validate configuration files and localize misconfigured parameter from logs
- **✗** Only work in certain cases – we'll discuss next
- [24'ICSE] Ciri, [24'Arxiv] CAIP, [24'Arxiv] SISDetector, [25'ISSTA] LogConfigLocalizer

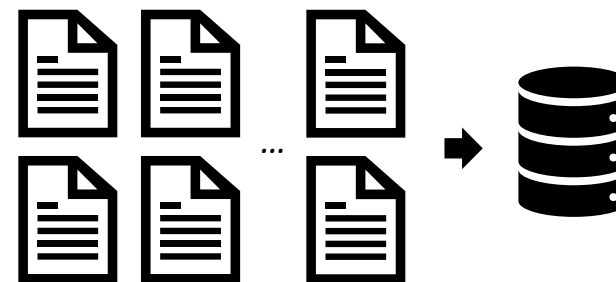
Limitations of Current LLM-based Approaches

Current LLM-based approaches have two fundamental limitations.

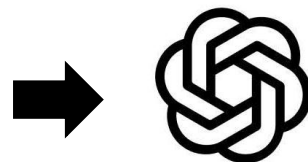
- First, they only work when the input contains recognizable clues.

System generate large amounts of logs that capture the runtime behavior of the system.

```
2025-03-25 08:21:46,364 INFO org.apache.hadoop.yarn.server.nodemanager.NodeManager: registered UNIX signal handlers for [TERM, HUP, INT]
2025-03-25 08:21:46,659 INFO org.apache.hadoop.yarn.server.nodemanager.containermanager.resourceplugin.ResourcePluginManager: The pluggable device framework is not enabled. If you want, please set true to yarn.nodemanager.pluggable-device-framework.enabled
2025-03-25 08:21:46,818 INFO org.apache.hadoop.yarn.event.AsyncDispatcher: Registering class org.apache.hadoop.yarn.server.nodemanager.containermanager.container.ContainerEventType for class org.apache.hadoop.yarn.server.nodemanager.containermanager.ContainerManagerImpl$ContainerEventDispatcher
2025-03-25 08:21:46,818 INFO org.apache.hadoop.yarn.event.AsyncDispatcher: Registering class org.apache.hadoop.yarn.server.nodemanager.containermanager.application.ApplicationEventType for class org.apache.hadoop.yarn.server.nodemanager.containermanager.ContainerManagerImpl$ApplicationEventDispatcher
```



```
ERROR .*: Failed to start namenode.
Java.lang.IllegalArgumentException: Invalid attribute
value for hadoop.security.authentication.* ...
```

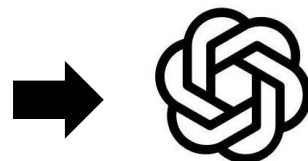


Misconfigured parameter:
hadoop.security.authentication



(a) If the log contains explicit clues, LLMs can successfully identify the misconfigured parameter.

```
INFO .*: Application report for application_.*
(state: ACCEPTED)
DEBUG .*: IPC Client (.*) connection to .* ...
```



Misconfigured parameter:
fs.defaultFS

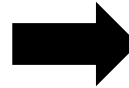


(b) If the log contains no explicit clues, LLMs fail to identify the misconfigured parameter.

Limitations of Current LLM-based Approaches

- The second limitation is hallucination.
 - Since contextual misconfigurations look syntactically correct, it is hard to tell from the configuration file alone whether a value is wrong or not.

```
...  
<property>  
  <name>hadoop.tmp.dir</name>  
  <value> file:///hadoop.txt </value>  
  <description>A base directories.</description>  
</property>  
<property>  
  <name>fs.defaultFS</name>  
  <value>hdfs://localhost:9000</value>  
  <description>A base directories.</description>  
</property>  
...
```



The misconfiguration is likely caused by 'fs.defaultFS' which appears to have an incorrect value...

The real culprit parameter is 'hadoop.tmp.dir'

An Example of LLM Hallucination

Our Approach

Falconf: Diagnosing misconfiguration via log sequence learning



Key insight

Logs contain latent patterns that reflect the underlying misconfiguration.



This means ...

Misconfigurations alter the flow of log events in distinctive ways, and by learning those patterns, we can identify which misconfiguration caused the failure.

Misconfiguration Log Analysis: Three Categories

We analyze how misconfigurations actually manifest in logs.

- We categorized misconfiguration-induced log outputs into 3 types based on how informative they are.
- **Type 1:** The log explicitly names the misconfigured parameter as the root cause.

```
ERROR .*: Failed to start namenode.  
Java.lang.IllegalArgumentException: Invalid attribute value for  
hadoop.security.authentication .* ...
```

(a) Log containing explicit clues – when 'hadoop.security.authentication' is set to a wrong value.

```
ERROR .*: java.lang.IllegalArgumentException: The value of  
spark.network.timeout=10000ms must be greater than the  
value of spark.executor.heartbeatInterval=10000ms.
```

(b) Log containing explicit clues – when 'spark.network.timeout' is set to an '10000ms'.

Misconfiguration Log Analysis: Three Categories

- **Type 2:** The log contains error messages, but no reference to any specific configuration parameter.

```
INFO .*: Receiving BP-.* src: .*, dest : .*  
INFO .*: Receiving BP-.* src: .*, dest : .*  
INFO .*: DIR -.* CompleteFile: .*
```

(a) Logs from normal execution

```
INFO .*: Receiving BP-.* src: .*, dest : .*  
INFO .*: ConnectException: Call from .*  
to .* failed on connection exception
```

The
deviating
part

(b) Logs from misconfiguration case –
Logs deviate from normal execution, with error messages

- **Type 3:** No error message at all. The application silently fails.

```
INFO .*: Application report ... (ACCEPTED)  
...  
INFO .*: Application report ... (RUNNING)  
INFO .*: Application report ... (FINISHED)
```

(a) Logs from normal execution

```
INFO .*: Application report ... (ACCEPTED)  
...  
INFO .*: Application report ... (ACCEPTED)  
DEBUG .*: IPC Client (.* ) connection to ...
```

The
deviating
part

(b) Logs from misconfiguration case –
Logs deviate from normal execution, with no error messages

Most Misconfiguration Leave No Explicit Clues in Logs

To quantify this, we analyze the misconfiguration-induced logs across three distributed systems.

- Systems: Spark, Hadoop and Zookeeper.

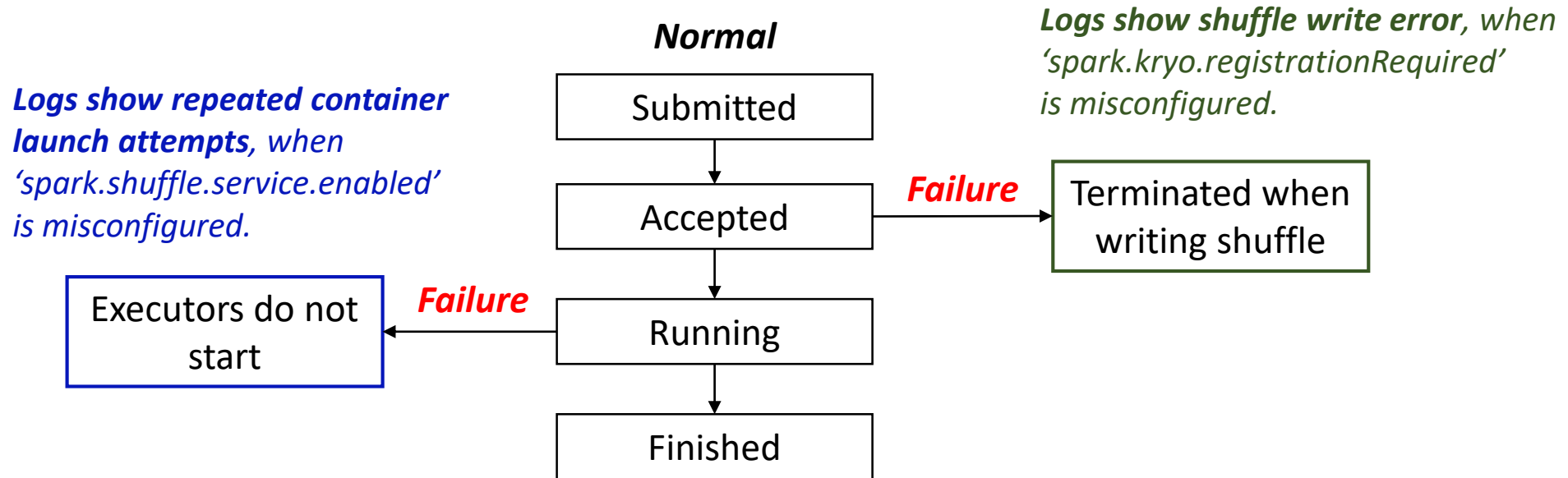
The majority of misconfiguration-induced failures produce logs with no explicit reference to the faulty parameter.

	Type 1	Type 2	Type 3
Spark	37.1%	60%	2.9%
Hadoop	10%	90%	0%
Zookeeper	0%	100%	0%

Distribution of failure cases by log type across three distributed systems

Each Misconfiguration Leaves a Distinctive Log Pattern

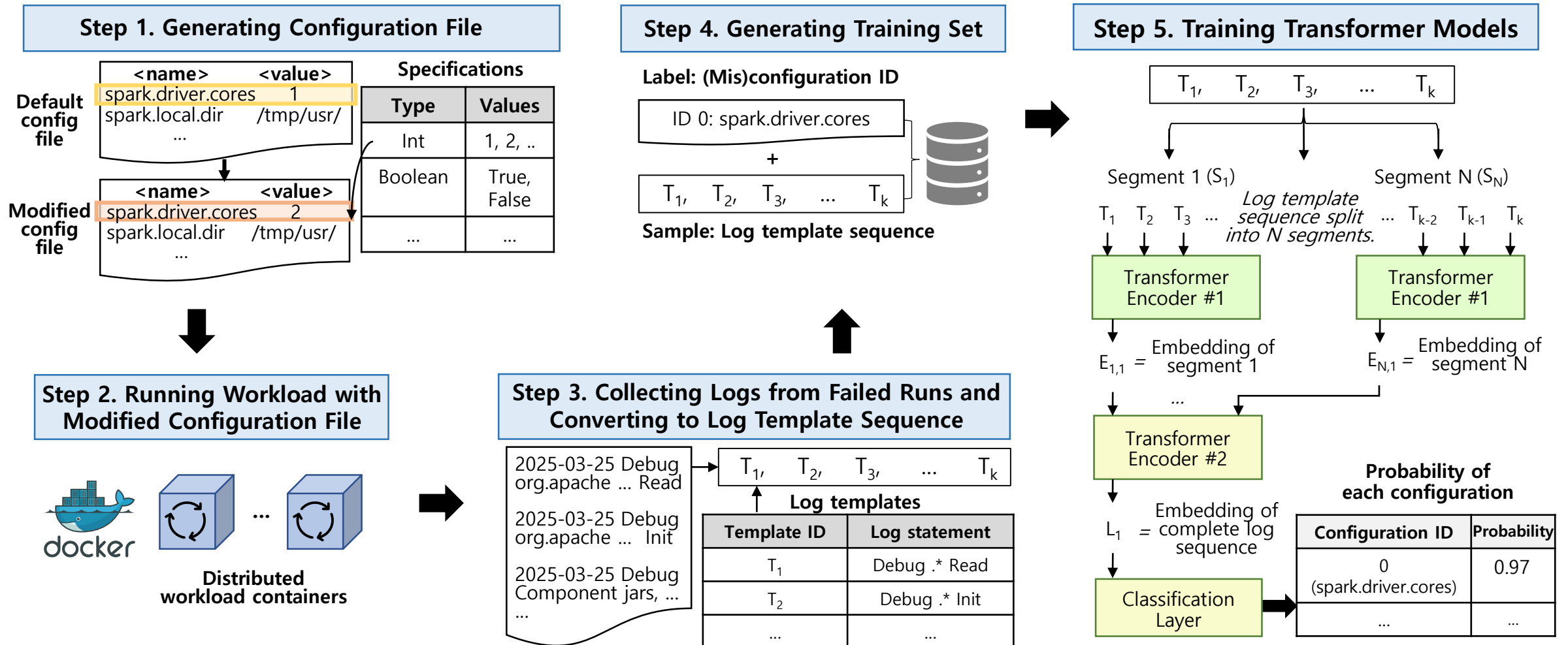
Each misconfiguration leaves a distinctive pattern in the sequence of log events.



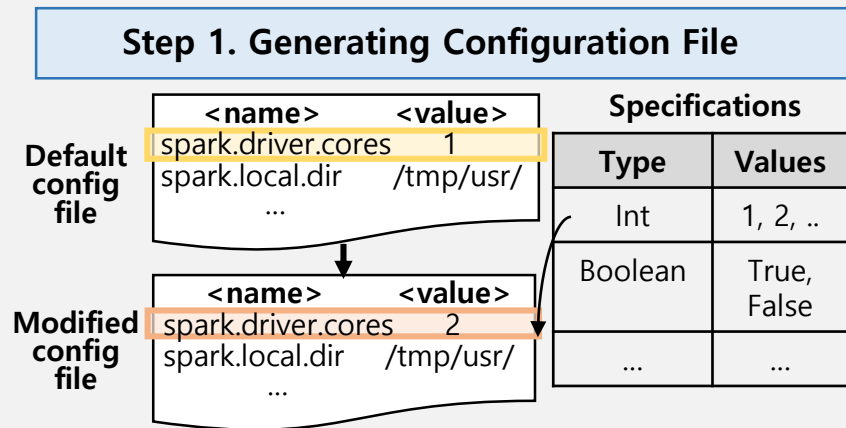
*Unique execution failure pattern per misconfigured parameter
(Workflow shown as a simplified representation of log sequence patterns)*

Our Proposed Framework

Architecture Overview



Step 1: Generating Misconfigured Configuration File



First, we generate misconfigured configuration file by injecting faulty values.

Step 2. Running Workload with Modified Configuration File



Step 3. Collecting Logs from Failed Runs and Converting to Log Template Sequence



Step 4. Generating Training Set

Label: (M)

ID 0:

$T_1, T_2, T_3, \dots, T_k$

Sample: Log template sequence

Step 5. Training Transformer Models

Segment 1 (S_1)

Segment N (S_N)

Log template sequence split into N segments.

$T_1, T_2, T_3, \dots, T_{k-2}, T_{k-1}, T_k$

Transformer Encoder #1

Transformer Encoder #1

Embedding of segment 1

Embedding of segment N

...

Transformer Encoder #2

Embedding of complete log sequence

Classification Layer

Probability of each configuration

Configuration ID	Probability
0 (spark.driver.cores)	0.97
...	...

Step 1: Generating Misconfigured Configuration File

We define the specification and plausible values for each type.

- The injected values are syntactically correct, but can cause failures at runtime.

```
... Spark configuration
<property>
  <name>spark.driver.cores</name>
  <value>1</value>
</property>
...
```



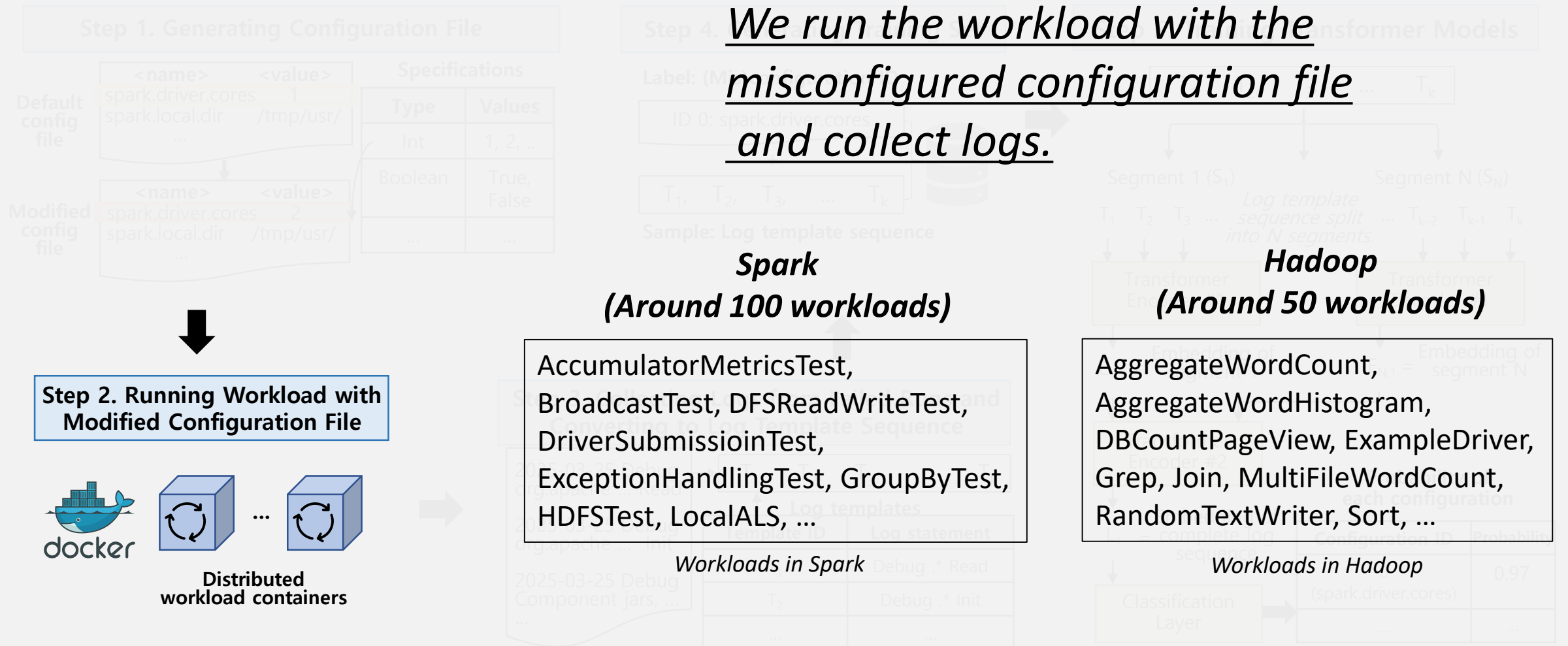
```
... Spark configuration
<property>
  <name>spark.driver.cores</name>
  <value>2</value>
</property>
...
```

Configuration Type	Specification	Example of Value
Boolean	Boolean has two valid values.	True, False
IP Address	IPv4 address consists of four decimal numbers.	192.168.1.100, 172.17.1.1
Port	Port that falls into value 0~65535.	7077, 7078, 7079, 7080
Size	Numeric value followed by unit (b, k, m, g).	2k, 4k, 256k, 4m, 2g, 8g, 512g
Int	Numeric value.	1, 2, 8, 10, 128, 100000
Time Interval	Numeric value followed by time unit (ms, s, m).	50ms, 1s, 2s, 2m, 5m, 30m
Path	Path consists of string and '/'.	File:///home/user/code.zip
String	Sequence of characters, including letters, numbers.	Wordcount, SHA-256
Float	Floating point numbers represented with decimal	0.2, 0.5, 0.7, 2.0, 5.0
URL	URL including scheme ("http") and domain name	http://proxy-example.com

Misconfiguration injected configuration file

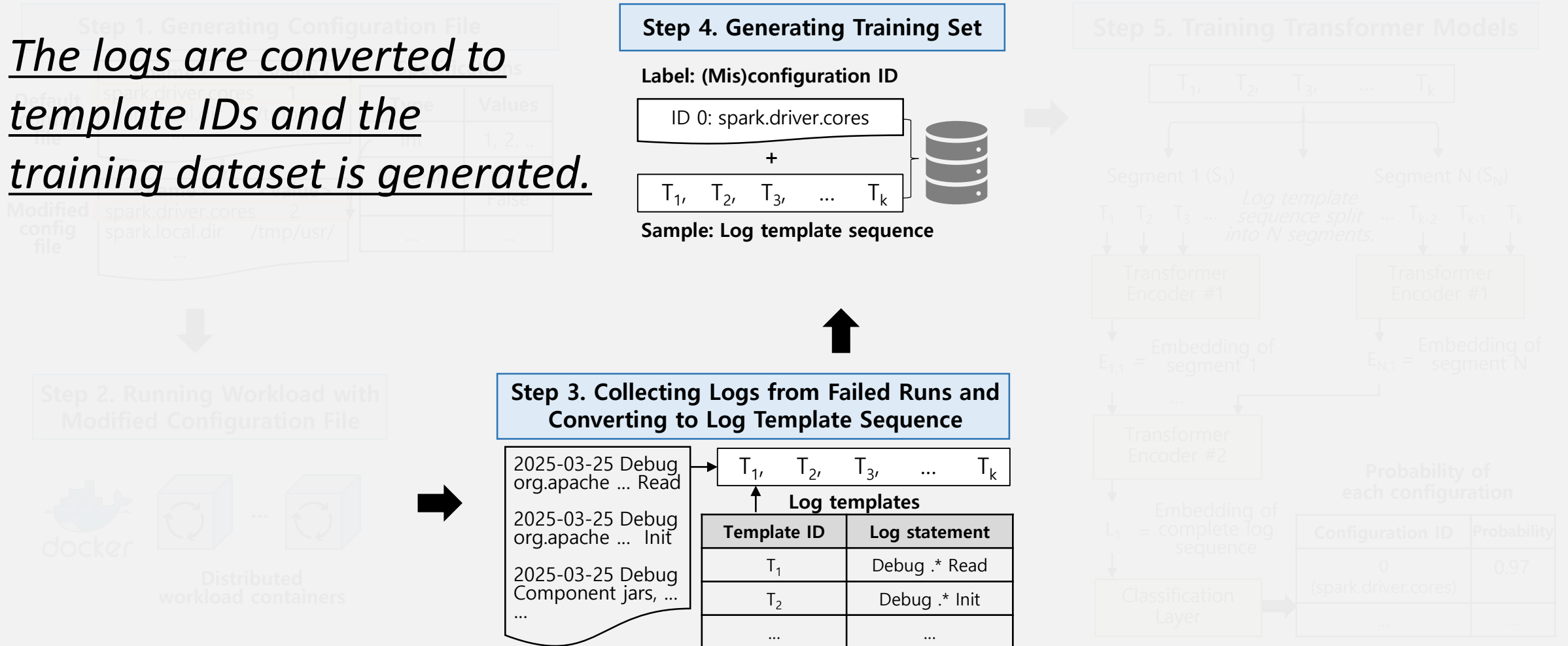
Specification per configuration type

Step 2: Run Workload to Collect Logs



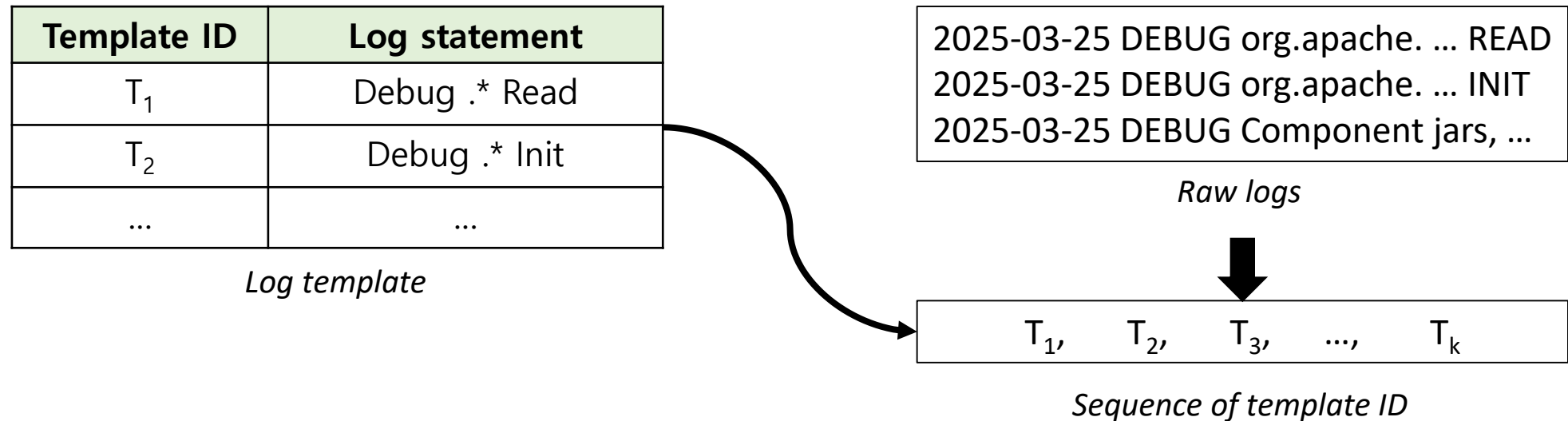
Step 3-4: Log Conversion & Dataset Construction

The logs are converted to template IDs and the training dataset is generated.



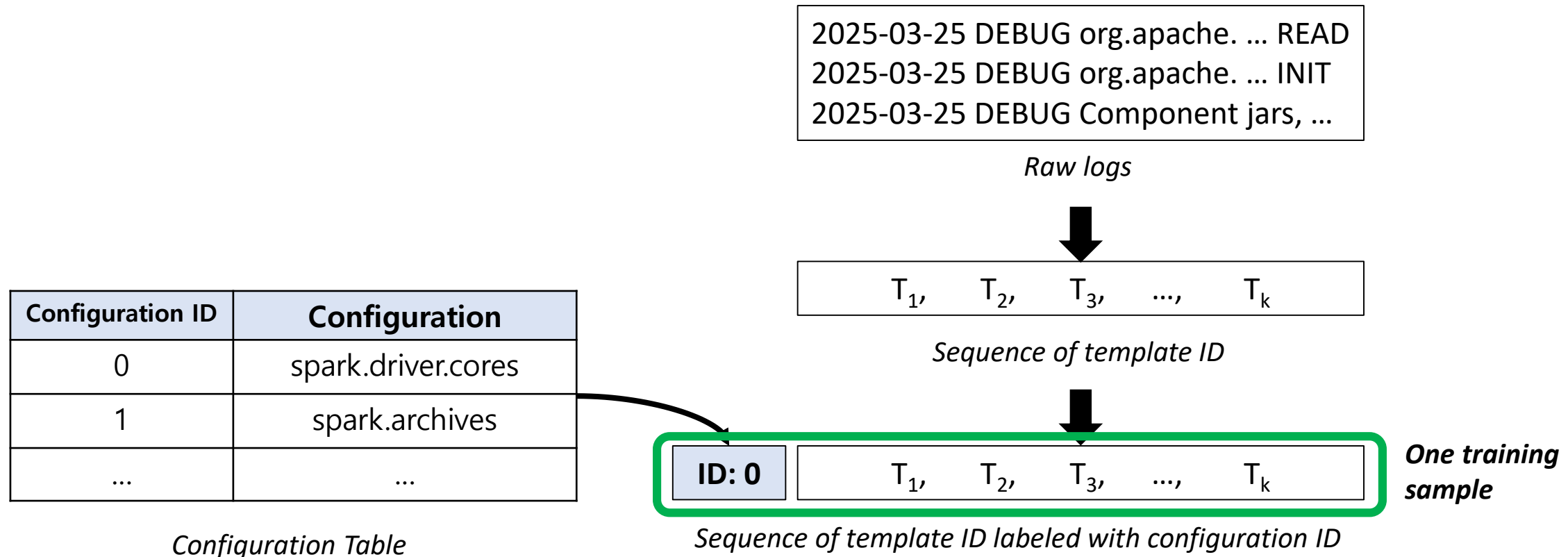
Step 3: Converting Raw Logs into Sequence of Log ID

Using the log template, raw logs are converted into sequence of log IDs.

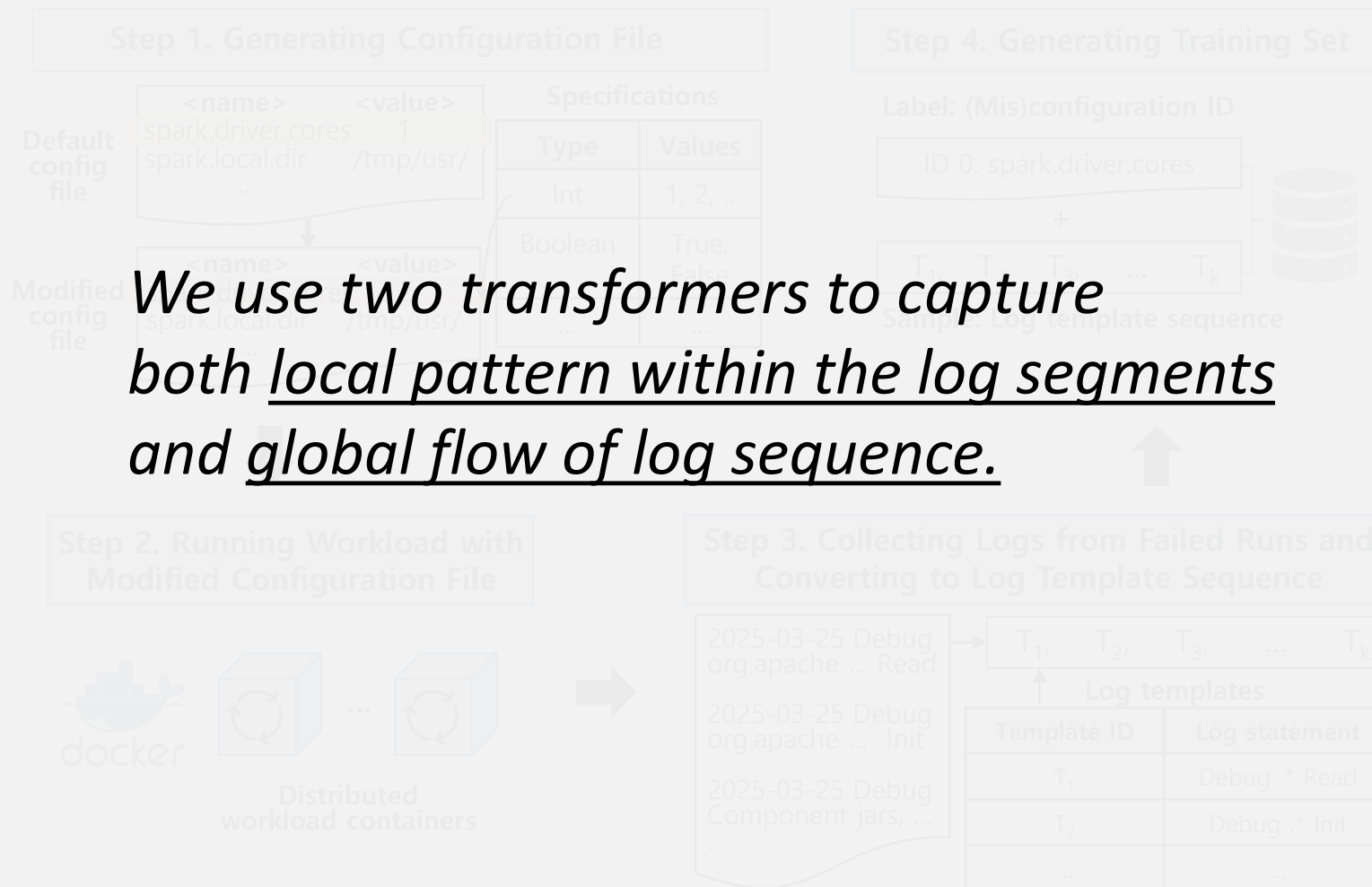


Step 4: Labeling with Configuration ID

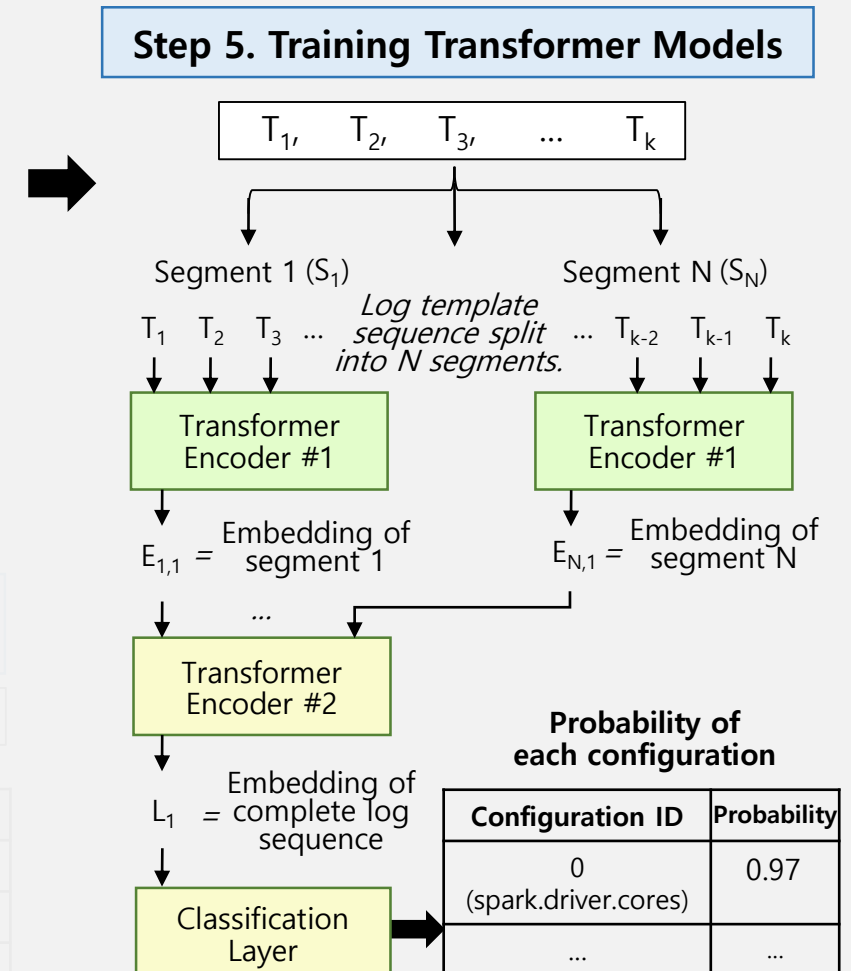
Then, each log template sequence is labeled with the ID of the misconfigured parameter, forming one training sample.



Step 5: Training Two-Stage Transformer Architecture

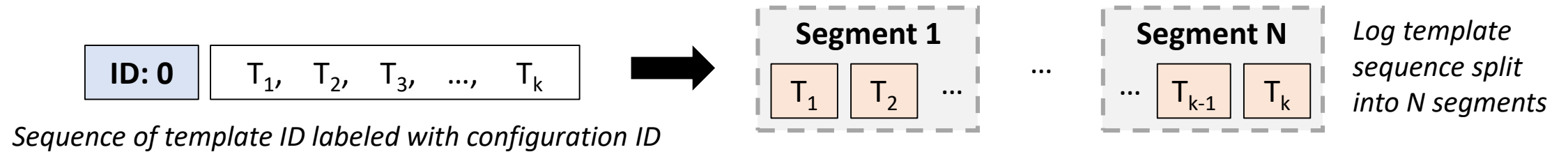


We use two transformers to capture both local pattern within the log segments and global flow of log sequence.



Step 5: Learning the Local Pattern in Log Segment

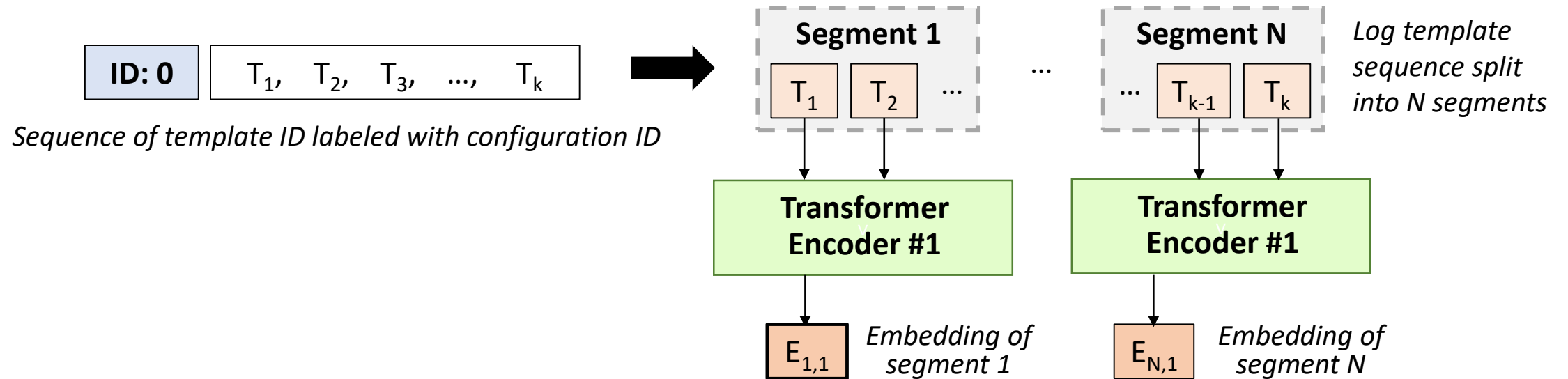
The log template sequence is split into N equal-sized segments.



Each segment is then independently processed by the first transformer encoder.

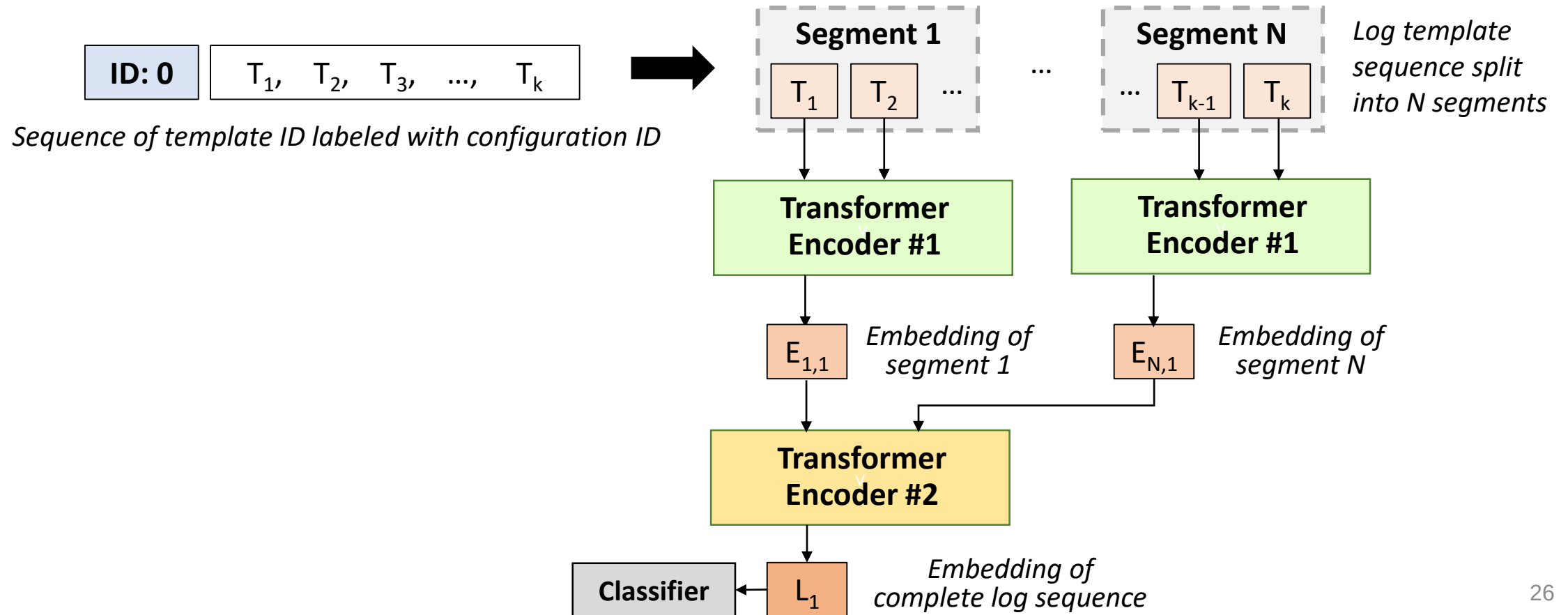
Step 5: Learning the Local Pattern in Log Segment

The first transformer produces embeddings for each segments.



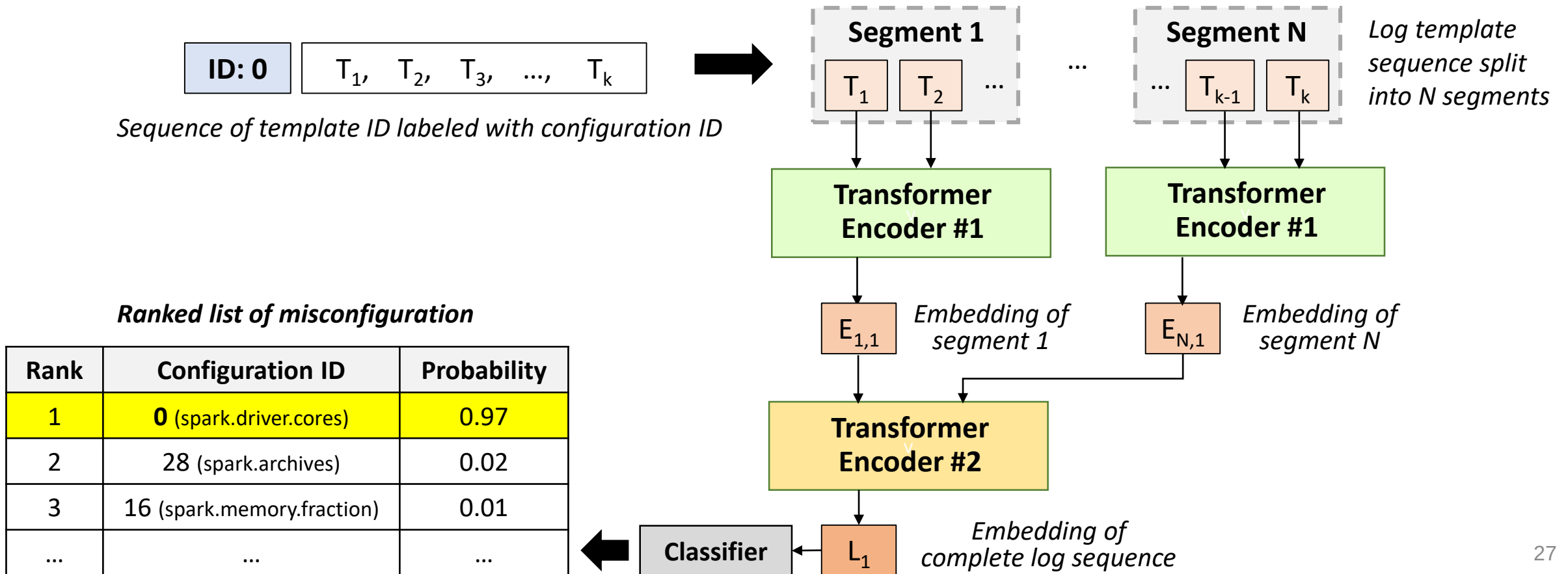
Step 5: Learning the Global Log Sequence Pattern

The second transformer captures the global flow of the entire log sequence.



Diagnosing Misconfiguration with Trained Model

At inference time, we use the output of classifier, which produces the ranked list of most likely misconfigured parameters.



Performance Evaluation

Evaluation

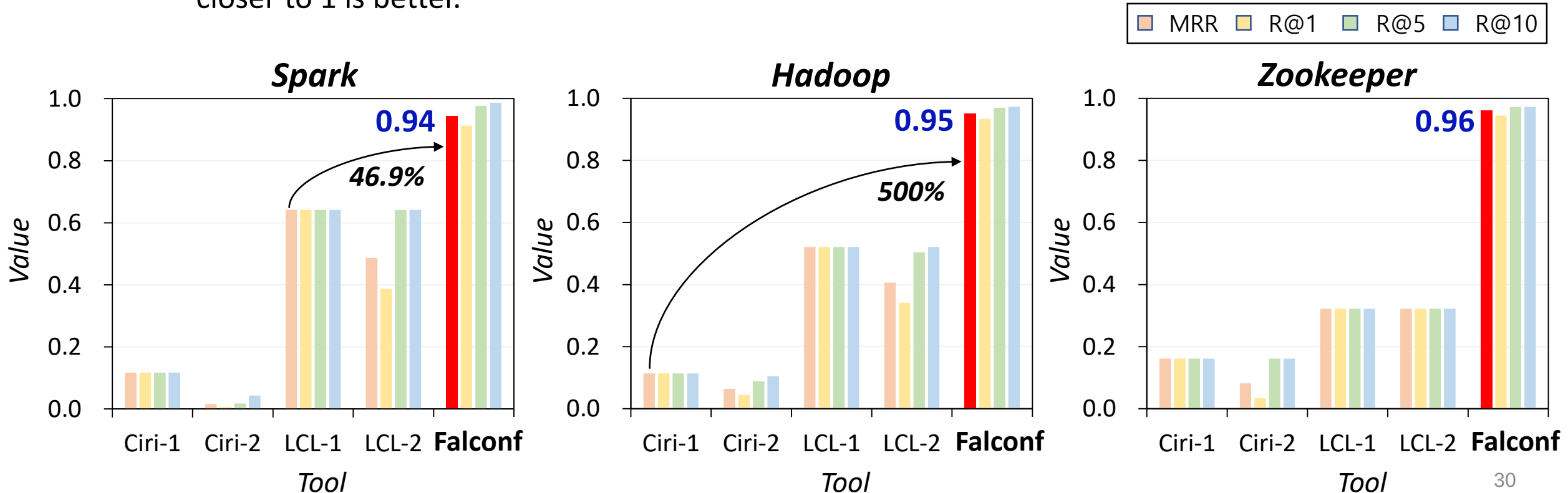
We evaluate Falconf from three perspectives.

1. Overall performance of Falconf (vs baselines).
2. Accuracy breakdown by misconfiguration log type.
3. Robustness across software version updates.

Falconf Achieves Average MRR of 0.95

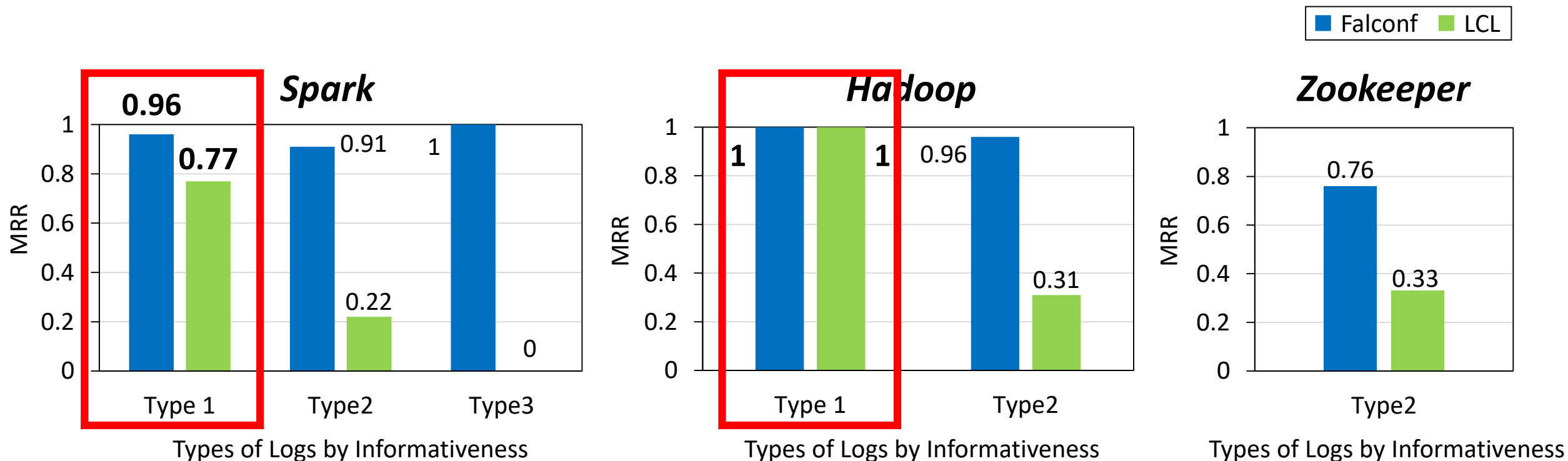
Falconf achieves the highest MRRs of 0.95 for all three target software, while others stays in 0.11 to 0.64.

- MRR: Measures how highly the correct misconfigured parameter is ranked, where a score closer to 1 is better.



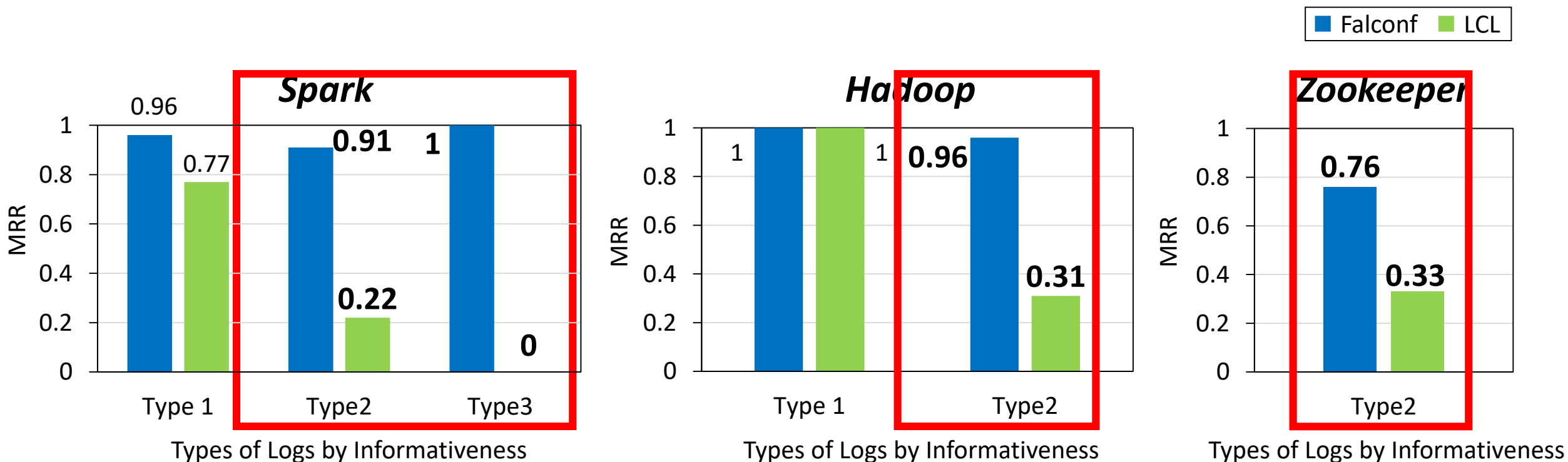
Falconf is Effective Regardless of Log Informativeness

For Type 1 cases, where the log explicitly names the misconfigured parameter, both Falconf and baselines show similar performance.



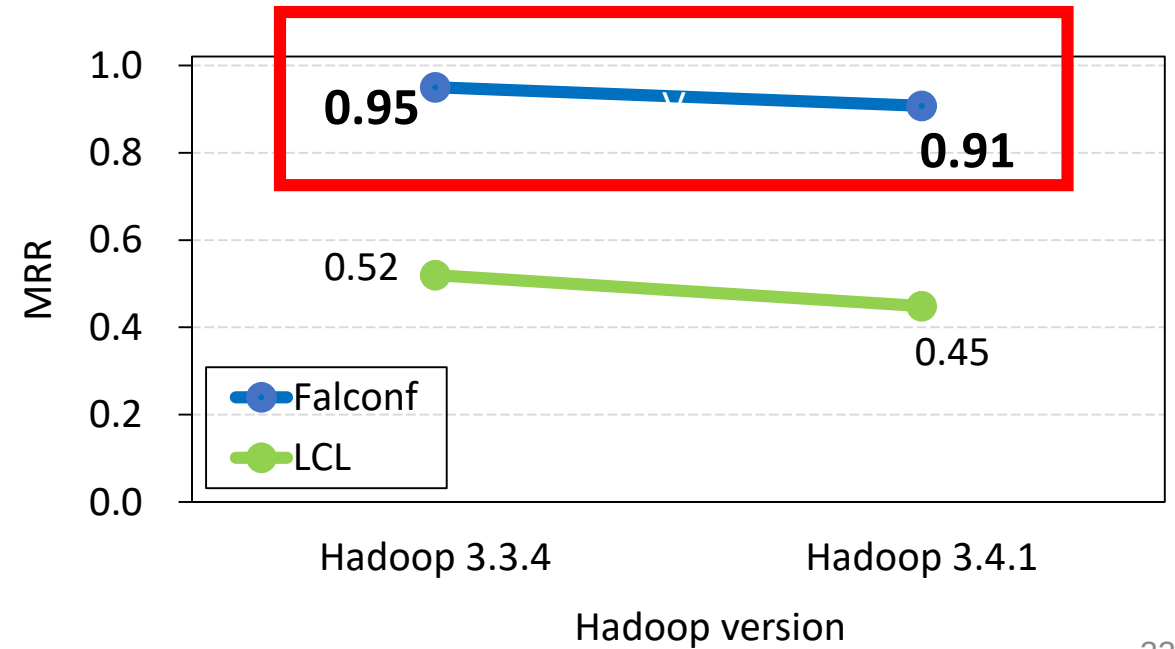
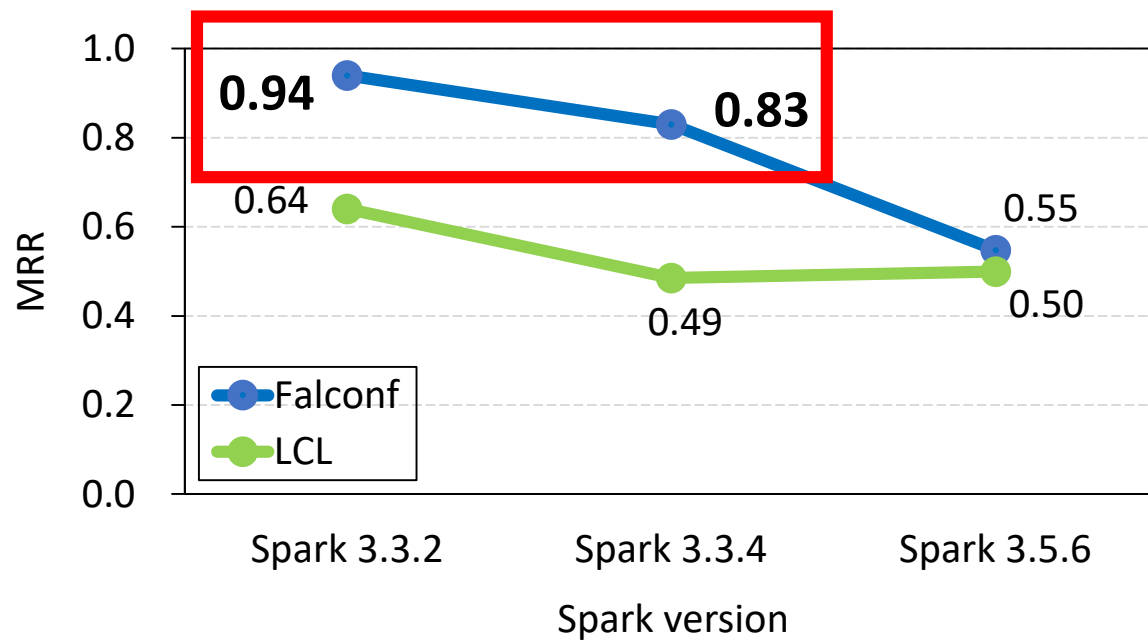
Falconf is Effective Regardless of Log Informativeness

Falconf outperforms in Type 2 and Type3, highlighting the advantage of learning from log sequence patterns when explicit or indirect hints to the culprit configuration parameters are absent.



Falconf is Robust to Incremental Version Updates

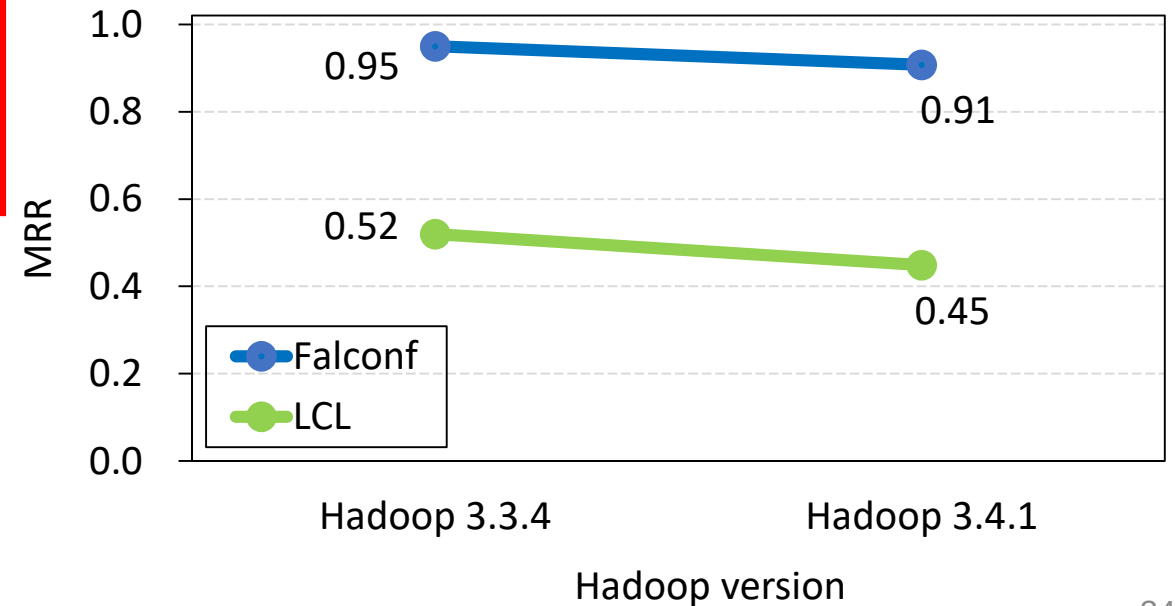
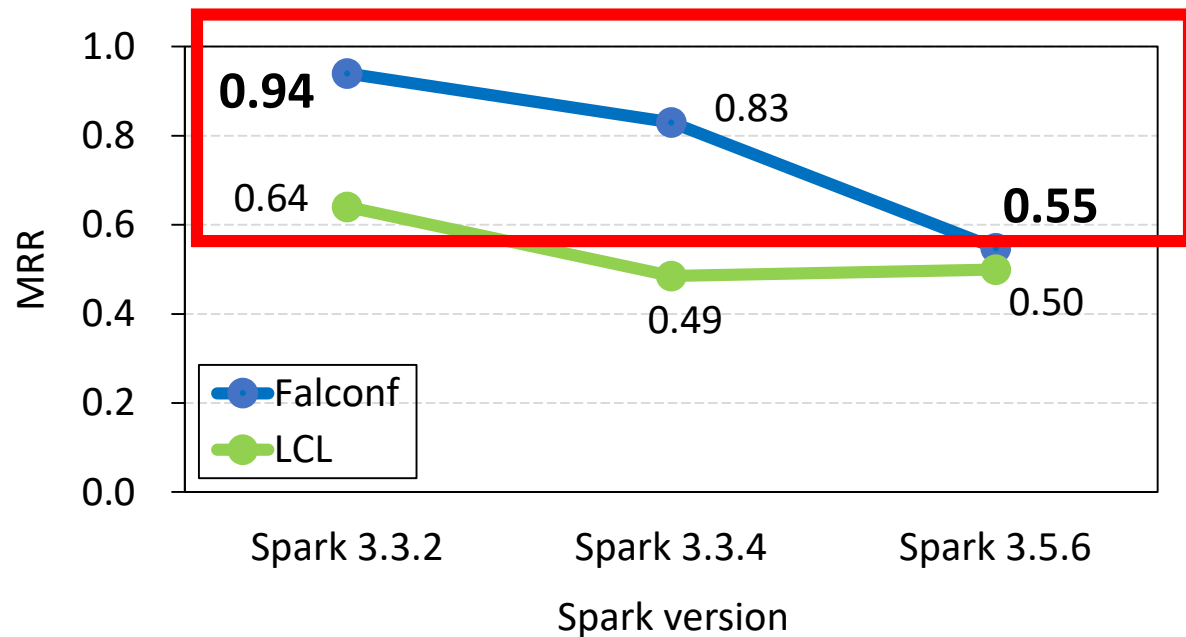
For incremental version changes, such as Spark 3.3.2 to 3.3.4, and Hadoop 3.3.4 to 3.4.1, Falconf maintains high accuracy.



Falconf May Require Retraining for Larger Version Jumps

However, for larger version jumps, accuracy drops from 0.94 to 0.55.

- This is because larger version jumps introduce changes in log message formats and reordering of execution stages, which alter log sequence pattern.



Conclusion

- We propose Falconf, which diagnoses misconfigurations via log sequence learning.
- Falconf automatically injects misconfigurations to generate training logs, and trains a two-stage transformer model on top of them.
- Falconf achieves an average MRR of 0.95, outperforming state-of-the-art LLM-based approaches by at least 46.9%.